

Reporte de Proyecto Final: Arquitectura, Diseño e Implementación de un Microprocesador MIPS Monociclo

INTRODUCCIÓN

El presente trabajo documenta la investigación, diseño e implementación a nivel de transferencia de registros (RTL) de un microprocesador MIPS (Microprocessor without Interlocked Pipelined Stages) de 32 bits de ciclo único. Históricamente, la arquitectura MIPS fue desarrollada a principios de la década de 1980 en la Universidad de Stanford, consolidándose como uno de los pilares fundamentales de la filosofía RISC (Reduced Instruction Set Computer). Diversas investigaciones y proyectos académicos a nivel global utilizan el diseño monociclo de MIPS como herramienta educativa principal debido a que su estructura sin segmentación (pipeline-less) facilita enormemente la comprensión visual y teórica de las fases de ejecución de una instrucción: Búsqueda, Decodificación, Ejecución, Acceso a Memoria y Escritura.

El marco teórico de esta práctica se sustenta en la abstracción de la interfaz entre el hardware y el software. Para comprender la simulación, es fundamental definir el datapath (o ruta de datos), que es el conjunto de componentes que procesan o almacenan datos, tales como la Unidad Aritmético Lógica (ALU), el Banco de Registros de 32 bits y las memorias. A la par del datapath, existe la Unidad de Control, que es la lógica combinacional encargada de interpretar el código de operación (opcode) de cada instrucción para emitir las señales eléctricas que dirigen el flujo de los multiplexores y los permisos de escritura. El diseño se fundamenta en la arquitectura Harvard, lo cual implica tener memorias separadas y dedicadas exclusivamente una para instrucciones y otra para datos, permitiendo leer una instrucción y acceder a los datos en el mismo ciclo de reloj.

Desde el punto de vista matemático y de rendimiento, el diseño de ciclo único se rige por una métrica donde los Ciclos por Instrucción (CPI) son estrictamente iguales a 1. Sin embargo, la teoría establece que la frecuencia máxima del reloj está severamente limitada por la instrucción más lenta. La fórmula matemática que rige el periodo del reloj en esta arquitectura es:

$$\text{Clock cycle} = \max(\text{delays})$$

Donde el retardo máximo suele estar dictado por la instrucción de carga de memoria (lw), cuyo camino crítico atraviesa la memoria de instrucciones, el banco de registros, la ALU, la memoria de datos y el multiplexor de escritura.

OBJETIVO

Objetivo General

Diseñar, modelar y verificar el funcionamiento de un procesador MIPS monociclo de 32 bits utilizando exclusivamente el Lenguaje de Descripción de Hardware Verilog, garantizando la correcta ejecución de un subconjunto específico de instrucciones RISC mediante simulaciones de flujo de bits.

Objetivos Particulares

1. **Desarrollar los módulos del Datapath:** Codificar en Verilog los módulos estructurales, incluyendo la ALU, el banco de registros (32 registros de 32 bits), el Contador de Programa (PC) y las memorias.
2. **Sintetizar la Unidad de Control:** Implementar la lógica combinacional de decodificación que gobierna las señales de control del sistema basándose en el "MIPS Core Subset".
3. **Comprobación Estructural:** Crear bancos de pruebas (Testbenches) en Verilog para validar la interacción de todos los módulos acoplados en el diseño *top-level*.

Alcances y Limitaciones

El alcance de la actividad contempla el desarrollo del módulo central del procesador (Top-Level) y sus submódulos internos. El procesador será capaz de ejecutar un subconjunto Turing-completo de instrucciones que incluye: operaciones aritmético-lógicas tipo R (add, sub, and, or, slt), operaciones de transferencia de datos tipo I (lw, sw), operaciones aritméticas con inmediatos (addi), y saltos de control de flujo (beq, j).

La limitación principal radica en la naturaleza del hardware de ciclo único; al carecer de segmentación (pipelining), su eficiencia temporal es menor en comparación con procesadores modernos, y el hardware combinacional de la ALU permanece inactivo durante los retardos de las memorias. Además, el diseño no contempla el manejo de excepciones, interrupciones o coprocesadores de punto flotante.

DESARROLLO

La metodología aplicada para la realización de este procesador siguió un enfoque de diseño modular ascendente (bottom-up) utilizando Verilog HDL. A continuación se detalla cómo se construyó cada módulo, los elementos utilizados y las justificaciones de diseño.

1. Módulo del Banco de Registros (Register File):

Se implementó como una matriz de memoria RAM en Verilog para albergar 32 registros temporales de 32 bits. Se utilizó lógica asíncrona para los dos puertos de lectura (basados en los campos rs y rt de la instrucción), y lógica síncrona mediante un bloque always @(posedge clk) para el puerto de escritura. Se forzó por diseño que el registro 0 (\$zero) siempre retorne el valor 0x00000000, una convención obligatoria en MIPS.

2. Módulo de la Unidad Aritmético Lógica (ALU):

La ALU es el núcleo de cálculo. Se codificó utilizando un bloque combinacional always

@* acoplado a una sentencia case en Verilog. Recibe dos operandos de 32 bits y una señal de control de 3 bits (ALUControl). Dependiendo de esta señal, la ALU realiza operaciones de suma, resta, AND, OR, o evaluación relacional (SLT). Además, se implementó una salida booleana Zero que se pone en alto (1) si el resultado de la sustracción es nulo, vital para la instrucción beq.

3. Memorias de Instrucciones y de Datos:

Se modelaron dos memorias independientes. La Memoria de Instrucciones actúa como una ROM asíncrona; se inicializó utilizando la directiva de Verilog \$readmemh para cargar el código máquina en formato hexadecimal directamente en el arreglo lógico. La Memoria de Datos se modeló como una RAM síncrona para la escritura (controlada por la señal MemWrite) y asíncrona para la lectura (controlada por MemRead), permitiendo las instrucciones sw y lw.

4. Módulos Auxiliares (Extensión de Signo y Sumadores):

Se programó un módulo combinacional para tomar los 16 bits menos significativos de las instrucciones tipo I y replicar su bit de signo para convertirlos a 32 bits. Asimismo, se implementaron sumadores dedicados puramente combinacionales (assign out = a + b) para calcular el avance secuencial del PC ($PC + 4$) y el cálculo de la dirección objetivo de las bifurcaciones.

5. Unidad de Control Principal y Decodificador ALU:

Para orquestar el datapath, se programó un decodificador que lee los 6 bits más significativos de la instrucción (opcode). Mediante asignaciones continuas y tablas de verdad implementadas en código Verilog, este módulo emite las señales maestras: RegDst, RegWrite, ALUSrc, MemtoReg, MemWrite, MemRead, Branch y Jump. Un sub-decodificador de la ALU toma la señal maestra ALUOp y, para las instrucciones tipo R, lee el campo funct (los 6 bits menos significativos) para dictaminar la operación exacta de la ALU.

6. Integración (Top-Level) y Comprobación (Testbench):

Una vez finalizados los módulos, se creó un módulo superior donde se instanciaron y conectaron internamente mediante cables (wire en Verilog). Para demostrar que el procesador funciona correctamente, se desarrolló un módulo de prueba (Testbench). Este programa no sintetizable en Verilog genera una señal de reloj oscilante (clk) y un pulso inicial de reinicio (reset). A través del simulador, se inyectaron instrucciones precompiladas en la memoria y se analizó el diagrama de tiempos (Waveform Analysis). Se trazó el flujo de bits comprobando que, por ejemplo, en un ciclo donde operaba un add, las señales RegWrite se activaban en 1 y la memoria de datos era ignorada, mientras que en un sw, el flujo de la ALU se dirigía exitosamente hacia el bus de dirección de la memoria RAM.

CONCLUSIÓN

Oscar Ramirez: creo que es una actividad sumamente desafiante, tanto en la lógica como en la practica ya haciendo el codigo, me gusto la complejidad solo que me falto mas explicacion sobre el tema final, pero en general me gusto mucho la clase y sus trabajos ya que es muy facil

aprender en esta clase o por lo menos para mi por que me interesan mucho los temas abordados.

Alondra Hernandez: Este proyecto me esta gustando mucho realizar porque me llama la atencion el hardware y este trabajo me permite acercarme a saber mas sobre como hacer un procesador.. Ademias, me permite elegir que componentes y que rapidez quiero que tenga a traves de ModelSim programando.

REFERENCIAS

1. Patterson, D. A., & Hennessy, J. L. (2004). *Computer Organization and Design: The Hardware/Software Interface* (3rd ed.). Morgan Kaufmann.
2. Karthika, K., Tarappa, Manoj, H. P., & Shylaja, V. (s.f.). *Implementation of MIPS (Processor)*. Bangalore Institute of Technology, International Journal of Engineering Research & Technology (IJERT).
3. Gurrola Navarro, M. A., Avelar Huerta, Á. E., & Bonilla Barragán, C. A. (2022). *Diseño de Procesador RISC-V de 32-Bits de Ciclo Único*. Pistas Educativas, 143. Tecnológico Nacional de México en Celaya.